

DSA STUDY BLUEPRINT SERIES

24. Strings Part3 Permutation in String

Checking if String Contains a Permutation of Substring

Section 04 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Sliding Window:** Maintain a window of length equal to target substring.
- **Hashing:** Match character frequency counts inside the window.

Memory & Execution Blueprint

Sliding Window Range (Target length = 3):

s2: [e i d] b a c o o o

Window → **[e, i, d]** matching target permutation!

C++ Implementation Reference

Standard code layout and syntax

```
bool checkInclusion(string s1, string s2) {  
    if (s1.length() > s2.length()) return false;  
    vector c1(26, 0), c2(26, 0);  
    for (int i = 0; i < s1.length(); i++) {  
        c1[s1[i] - 'a']++; c2[s2[i] - 'a']++;  
    }  
    for (int i = s1.length(); i < s2.length(); i++) {  
        if (c1 == c2) return true;  
        c2[s2[i] - 'a']++; c2[s2[i] - s1.length() - 'a']--;  
    }  
    return c1 == c2;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N + M)$
Space Complexity	$O(1)$ (Fixed size counts)

Key Practices & Gotchas

- **Important:** The frequency array size is always 26, guaranteeing $O(1)$ auxiliary space.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run two-pointer string validations or window updates for string "race" :

Step	Left Pointer Index	Right Pointer Index	Char Comparison	Outcome
1	left = 0 ('r')	right = 3 ('e')	'r' != 'e'	Validation stops (Not a palindrome)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Sliding Window Patterns:** Track character frequency maps within a moving boundary window size to find substring anagrams and permutations.
- **In-place reversals:** Swap characters inside word boundaries using double pointer offsets.
- **Related LeetCode Problems:** #3 Longest Substring Without Repeating Characters, #567 Permutation in String, #344 Reverse String.